

Το 1ο σετ ασκήσεων αφορά τον παράλληλο προγραμματισμό με OpenMP. Στην εργασία αυτή μου ζητείται η παραλληλοποίηση της εύρεσης πρώτων αριθμών με χρήση `parallel for`, η παραλληλοποίηση της ταξινόμησης με `merge sort` και `tasks` και τέλος μια σύντομη ανάλυση των εννοιών `final` και `mergeable`.

Σύστημα εκτέλεσης

Όνομα υπολογιστή	opti323.vlan74.mgm
Επεξεργαστής	Intel Core i3-8300 @ 3.70GHz
Πλήθος πυρήνων	4
Μεταφραστής	gcc 11.2.0 (-O2 -fopenmp)

Άσκηση 1 – Παραλληλοποίηση εύρεσης πρώτων αριθμών (40%)

Το πρόβλημα

Αρχικά μου δίνεται η συνάρτηση `serial_primes()` η οποία, για γνωστό N , υπολογίζει πόσοι πρώτοι αριθμοί υπάρχουν μέχρι και το N και βρίσκει τον μεγαλύτερο από αυτούς. Οπότε αυτό που θα κάνω είναι η υλοποίηση της `openmp_primes()` με τους ίδιους υπολογισμούς δηλαδή χωρίς να αλλάξω τον αρχικό αλγόριθμο αλλά τώρα θα γίνει παράλληλα.

Μέθοδος παραλληλοποίησης

Ο κύριος βρόχος `for` της `serial_primes()` επαναλαμβάνεται ανεξάρτητα για κάθε περιττό αριθμό, οπότε μπορεί να παραλληλοποιηθεί. Στην OpenMP έκδοση χρησιμοποίησα την εντολή `#pragma omp parallel for` με `schedule(runtime)`. Έτσι για κάθε τύπο διαμοιρασμού, γράφω κατά την εκτέλεση στο terminal το `OMP_SCHEDULE` και κερδίζω στο ότι κάθε φορά δεν χρειάζεται να είναι σταθερά γραμμένη μέσα στον κώδικα. Δοκίμασα τις πολιτικές `static`, `guided` και `dynamic`. Τις μεταβλητές `num`, `divisor`, `quotient` και `remainder` τις δήλωσα ως `private`, ώστε κάθε νήμα να δουλεύει με τις δικές του τιμές. Για το πλήθος των πρώτων χρησιμοποίησα το `reduction(+)`, ενώ για τον μεγαλύτερο πρώτο χρησιμοποίησα `reduction(max)` για να μην προκύπτουν `race conditions`.

Οπότε για να μην αλλάζω συνέχεια τον κώδικα στο terminal γράφω :

```
export OMP_SCHEDULE=static
```

```
./primes
```

```
export OMP_SCHEDULE=guided
```

```
./primes
```

```
export OMP_SCHEDULE=dynamic
```

```
./primes
```

Επιλογή schedule

Δοκιμάσα τρεις πολιτικές διαμοιρασμού επαναλήψεων:

-static: κάθε νήμα παίρνει από πριν ένα σταθερό τμήμα επαναλήψεων. Έχει μικρό overhead και αυτό μπορεί να οδηγήσει σε άνιση κατανομή φόρτου μεταξύ των νημάτων.

-guided: ο scheduler ξεκινά με μεγαλύτερα chunks και στη συνέχεια τα μικραίνει, προσπαθώντας να ισορροπήσει ανάμεσα σε καλύτερο load balancing και μικρότερο overhead.

-dynamic: οι επαναλήψεις μοιράζονται δυναμικά σε μικρότερα chunks, ώστε τα νήματα να παίρνουν νέα δουλειά καθώς ολοκληρώνουν την προηγούμενη.

Ο βρόχος εξετάζει περιττούς αριθμούς της μορφής $2i+3$. Καθώς μεγαλώνει ο αριθμός, αυξάνεται και το κόστος του ελέγχου πρωτότητας, οπότε είναι πιθανό να εμφανιστεί άνιση κατανομή φόρτου. Για αυτό δοκιμάστηκαν διαφορετικές πολιτικές και επιλέχθηκε τελικά εκείνη που έδωσε τα καλύτερα αποτελέσματα στις μετρήσεις.

Πειραματικά αποτελέσματα – μετρήσεις

Χρησιμοποίησα το `omp_get_wtime()` για χρονομέτρηση. Κάθε πείραμα εκτελέστηκε 4 φορές και υπολογίστηκε ο μέσος όρος. Τιμή $N = 10.000.000$.

Πίνακας 1: Χρόνοι εκτέλεσης primes – schedule(static) (sec)

Νήματα	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
serial	6.493	6.493	6.493	6.492	6.493
1	6.499	6.500	6.498	6.500	6.499
2	4.058	4.058	4.058	4.058	4.058
3	2.836	2.836	2.837	2.836	2.836
4	2.174	2.174	2.173	2.173	2.173

Πίνακας 2: Χρόνοι εκτέλεσης primes – schedule(guided) (sec)

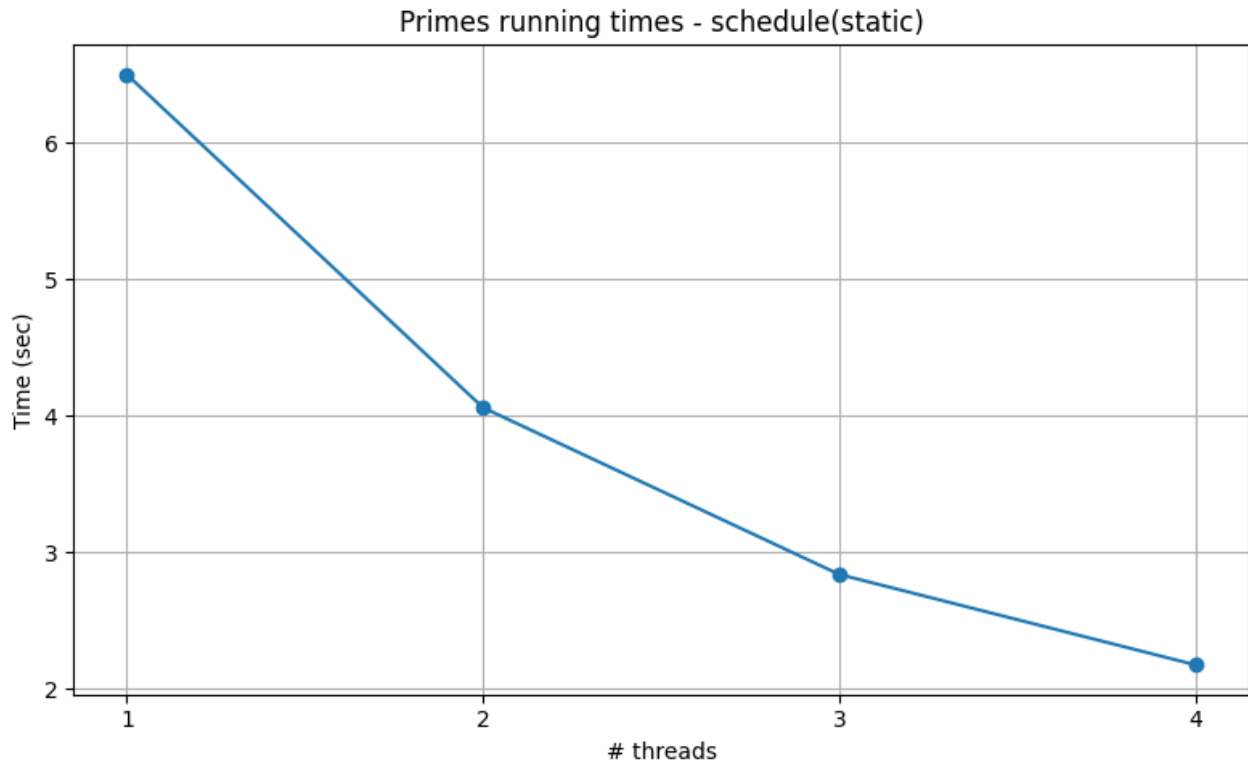
Νήματα	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
serial	6.493	6.493	6.493	6.495	6.493
1	6.499	6.500	6.500	6.500	6.499
2	3.250	3.250	3.250	3.250	3.250
3	2.166	2.167	2.167	2.167	2.167
4	1.645	1.625	1.626	1.625	1.630

Πίνακας 3: Χρόνοι εκτέλεσης primes – schedule(dynamic) (sec)

Νήματα	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
serial	6.493	6.492	6.493	6.493	6.493
1	6.540	6.539	6.540	6.539	6.540
2	3.286	3.285	3.283	3.283	3.284
3	2.192	2.193	2.191	2.192	2.192
4	1.647	1.647	1.646	1.649	1.647

ΓΡΑΦΙΚΕΣ ΠΑΡΑΣΤΑΣΕΙΣ:

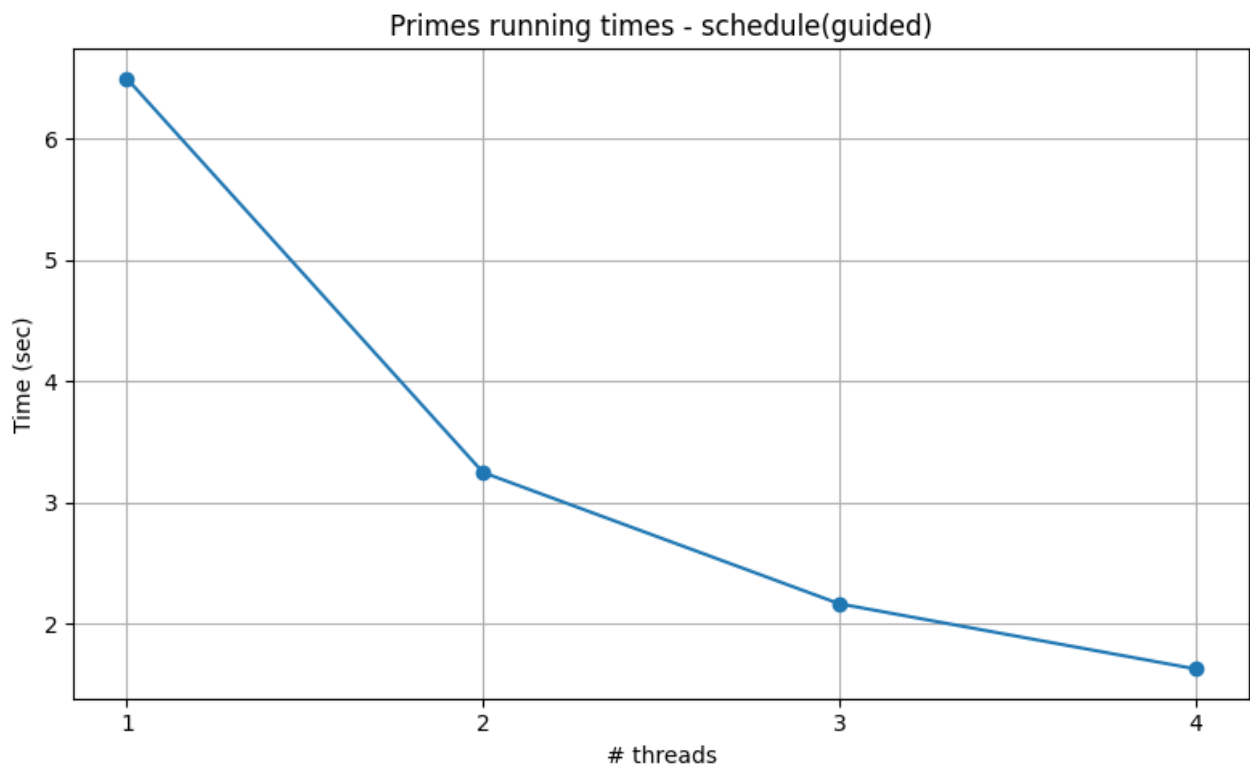
Για τον πίνακα 1:



Για static:

Από το σχήμα φαίνεται ότι ο χρόνος εκτέλεσης μειώνεται όσο αυξάνεται ο αριθμός των νημάτων. Με 1 νήμα η OpenMP έκδοση έχει σχεδόν ίδιο χρόνο με τη σειριακή, ενώ με 2, 3 και 4 νήματα εμφανίζεται σαφής βελτίωση. Όμως η πολιτική static δεν είχε και τις καλύτερες επιδόσεις σε σχέση με τις άλλες πολιτικές που θα εξετάσω παρακάτω. Αυτό είναι λογικό, γιατί οι επαναλήψεις δεν έχουν όλες το ίδιο υπολογιστικό κόστος. Έτσι, με στατικό διαμοιρασμό, μπορεί να προκύψει άνιση κατανομή φόρτου στα νήματα.

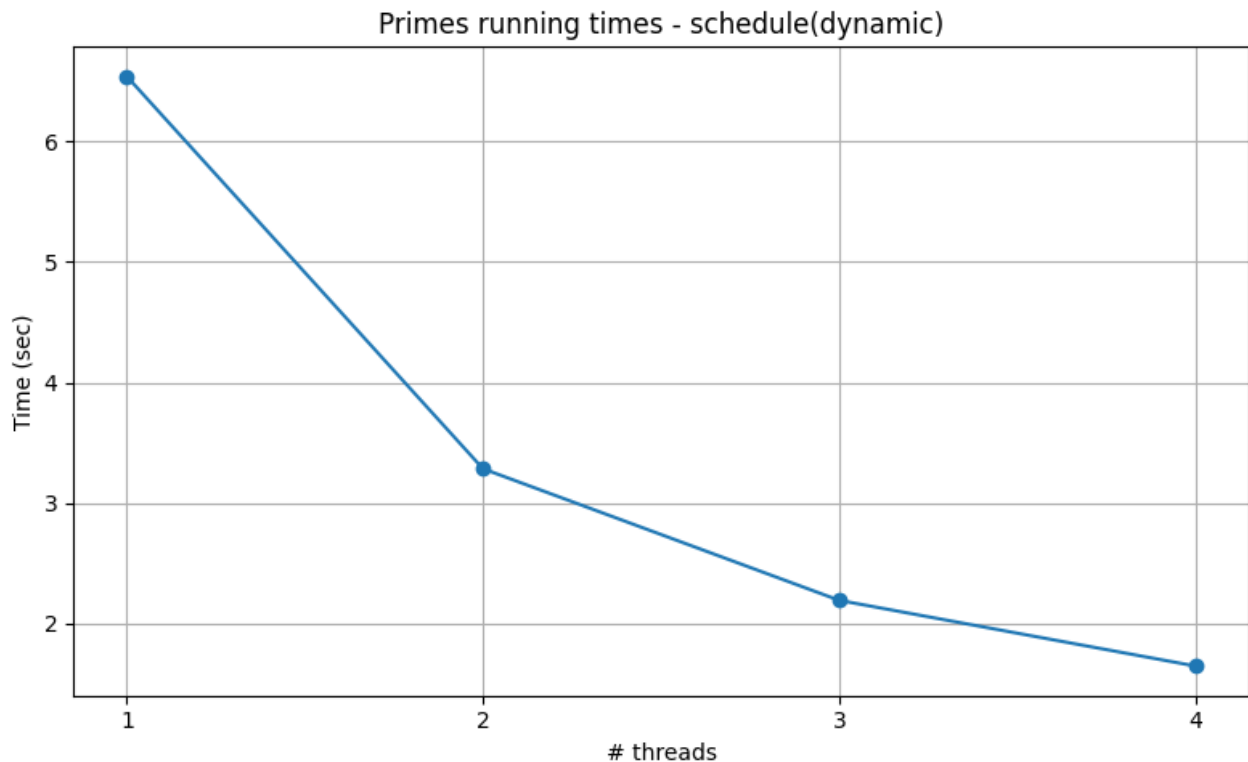
Πίνακας 2:



Για guided

Στην περίπτωση του guided, ο χρόνος εκτέλεσης μειώνεται σταθερά όσο αυξάνονται τα νήματα. Σε σχέση με το static, η πολιτική αυτή έδωσε καλύτερους χρόνους σε όλες τις παράλληλες εκτελέσεις και μάλιστα τη συνολικά καλύτερη επίδοση στα πειράματα. Είναι λογικό αφού το guided ξεκινά με μεγαλύτερα chunks και στη συνέχεια τα μικραίνει, πετυχαίνοντας καλύτερη ισορροπία ανάμεσα σε load balancing και μικρό overhead διαχείρισης.

Πίνακας 3:



Για dynamic

Από το σχήμα βλέπω ότι η πολιτική dynamic βελτιώνει επίσης τον χρόνο εκτέλεσης όσο αυξάνονται τα νήματα. Η συμπεριφορά της είναι καλύτερη από τη static, όμως στα συγκεκριμένα πειράματα ήταν λίγο πιο χειρότερη από την guided, κυρίως στα 3 και 4 νήματα. Αυτό δείχνει ότι ο δυναμικός διαμοιρασμός βοηθά στο load balancing, αλλά στη συγκεκριμένη περίπτωση το επιπλέον overhead διαχείρισης δεν μας οδήγησε στην καλύτερη συνολική επίδοση.

Σχόλια:

Από τα αποτελέσματα φαίνεται ότι η επιλογή του schedule επηρεάζει αισθητά τον χρόνο εκτέλεσης. Και οι τρεις πολιτικές βελτιώνουν την απόδοση όσο αυξάνεται ο αριθμός των νημάτων, όμως η συμπεριφορά τους δεν είναι ίδια. Η static ήταν η πιο αργή από τις τρεις, κάτι που αποδεικνύει ότι ο στατικός διαμοιρασμός δεν εκμεταλλεύεται το ίδιο καλά όλα τα νήματα όταν οι επαναλήψεις δεν έχουν ακριβώς ίδιο κόστος. Η dynamic είχε επίσης καλή συμπεριφορά και έδωσε σημαντική επιτάχυνση, αλλά στα συγκεκριμένα runs ήταν λίγο πιο χειρότερη από την guided. Συγκεκριμένα, στα 4 νήματα το guided είχε μέσο χρόνο 1.630 sec, ενώ το dynamic 1.647 sec και το static 2.173 sec, κάτι που επιβεβαιώνει ότι το guided ήταν η καλύτερη επιλογή στα συγκεκριμένα πειράματα. Τέλος για τα πειράματα που πραγματοποιήσα, η guided έδωσε την καλύτερη απόδοση και την επιλέγω ως την καταλληλότερη πολιτική για το Α ερώτημα.

Άσκηση 2 – Παραλληλοποίηση του merge sort με tasks (40%)

Το πρόβλημα

Δίνεται η σειριακή αναδρομική υλοποίηση της `mergesort_serial()`, ενώ στη συνέχεια ζητείται η υλοποίηση της `mergesort_parallel()` με χρήση `tasks` του OpenMP, ώστε τα δύο αναδρομικά μισά του πίνακα να ταξινομούνται ταυτόχρονα.

Μέθοδος παραλληλοποίησης

Σε κάθε επίπεδο αναδρομής, τα δύο μισά του πίνακα είναι ανεξάρτητα, οπότε μπορούν να ταξινομηθούν παράλληλα. Έτσι δημιουργώ δύο OpenMP `tasks`, ένα για κάθε υποπρόβλημα. Η εκτέλεση ξεκινά μέσα από μια `parallel` περιοχή και μια `single` εντολή στη `main()`, η οποία καλεί τη `mergesort_parallel()`.

```
#pragma omp parallel
{
    #pragma omp single
    mergesort_parallel(array, n, tmp);
}
```

Για μικρούς πίνακες ($n \leq \text{TASK_CUTOFF} = 8192$), η αναδρομή σταματά και εκτελείται η σειριακή εκδοχή ώστε να αποφεύγεται το overhead της δημιουργίας πολλών μικρών `tasks`.

Πειραματισμός με TASK_CUTOFF

Δοκιμάσα διαφορετικές τιμές του cutoff. Για πολύ μικρές τιμές δημιουργήθηκαν πάρα πολλά `tasks` και αυξάνουν το overhead, ενώ πολύ μεγάλες τιμές αφήνουν ανεκμετάλλευτο μέρος του παραλληλισμού. Στα συγκεκριμένα πειράματα, η τιμή `TASK_CUTOFF = 8192` μου έδωσε ικανοποιητική συμπεριφορά για πίνακα μεγέθους περίπου 20 εκατομμυρίων στοιχείων.

Πίνακας 4a: Χρόνοι εκτέλεσης Mergesort – tasks (sec)

TASK_CUTOFF	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
2048	0.849	0.896	0.847	0.846	0.860
8192	0.841	0.843	0.840	0.888	0.853
32768	0.888	0.888	0.889	0.887	0.888

Για να στηριχθεί η επιλογή του `TASK_CUTOFF` έκανα κάποιες επιπλέον δοκιμές με 4 νήματα για τις τιμές 2048, 8192 και 32768. Έτσι η τιμή 8192 έδωσε τον μικρότερο μέσο χρόνο εκτέλεσης στα συγκεκριμένα πειράματα. Οπότε την επέλεξα ως τιμή του `TASK_CUTOFF` στο πείραμα του Β ερωτήματος.

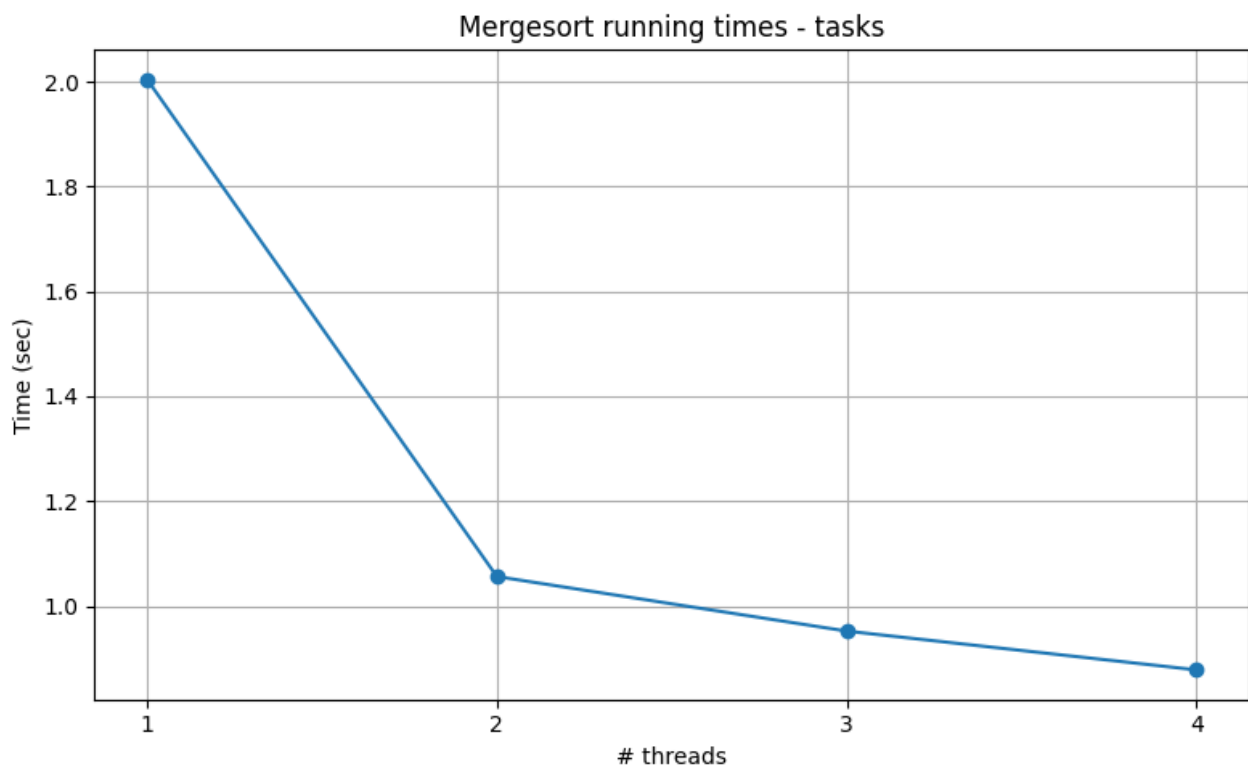
Πειραματικά αποτελέσματα – μετρήσεις

Μέγεθος πίνακα: 20.000.000 στοιχεία (όρισμα 20). Κάθε πείραμα εκτελέστηκε 4 φορές.

Πίνακας 4b: Χρόνοι εκτέλεσης Mergesort – tasks (sec)

Νήματα	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
serial	2.033	2.040	2.033	2.033	2.035
1	2.004	2.005	2.004	2.005	2.004
2	1.056	1.057	1.056	1.056	1.056
3	0.840	0.839	1.066	1.063	0.952
4	0.843	0.892	0.890	0.889	0.878

ΓΡΑΦΙΚΗ 4b:



Σχόλια:

Από τα αποτελέσματα συμπεραίνω ότι η χρήση tasks βελτιώνει τον χρόνο εκτέλεσης του merge sort όσο αυξάνεται ο αριθμός των νημάτων. Με 1 νήμα η παράλληλη έκδοση έχει σχεδόν ίδιο χρόνο με τη σειριακή που είναι λογικό αφού δεν υπάρχει πραγματικό κέρδος από την παραλληλία και παραμένει κυρίως το overhead των OpenMP tasks. Με 2 νήματα η βελτίωση είναι ήδη μεγάλη, ενώ με 3 και 4 νήματα ο χρόνος μειώνεται ακόμη περισσότερο. Στα συγκεκριμένα runs υπάρχει διακύμανση στην περίπτωση των 3 νημάτων αλλά η καλύτερη επίδοση είχα στα 4 νήματα. Αυτό είναι λογικό, γιατί τα δύο υποπροβλήματα της merge sort μπορούν να εκτελούνται παράλληλα με tasks, ενώ το cutoff βοηθά να μην υπάρχει μεγάλο overhead από τη δημιουργία πολλών μικρών tasks. Πειραματικά, ο μέσος σειριακός χρόνος ήταν 2.035 sec, ενώ με 4 νήματα ο μέσος παράλληλος χρόνος έπεσε στα 0.878 sec, δηλαδή η επιτάχυνση έφτασε περίπου το 2.32x.

Το merge όμως συνεχίζει να γίνεται σειριακά μετά το taskwait, κάτι που περιορίζει τη συνολική επιτάχυνση.

Άσκηση 3 – Φράσεις final και mergeable (20%)

Περιγραφή φράσεων (OpenMP 5.0)

final(expr)

Όταν η συνθήκη expr είναι αληθής, το task δημιουργείται ως final task. Στην πράξη αυτό σημαίνει ότι σε χαμηλότερα επίπεδα της αναδρομής αποφεύγεται η δημιουργία πολλών νέων deferred tasks και έτσι μειώνεται το overhead του scheduler. Στο merge sort αυτό είναι χρήσιμο όταν τα υποπροβλήματα έχουν ήδη μικρύνει αρκετά.

mergeable

Η φράση mergeable επιτρέπει στο runtime, όπου το θεωρεί κατάλληλο, να χειριστεί ένα task με πιο ελαφρύ τρόπο σε σχέση με το data environment του parent task. Δεν εγγυάται από μόνη της βελτίωση στην απόδοση, αλλά μπορεί να βοηθήσει στη μείωση κάποιου overhead. Για τον λόγο αυτό τεστάρετε σε συνδυασμό με το final.

Σύνδυασμός final + mergeable στο Mergesort:

Στην εργασία αυτή χρησιμοποιήσα final(n <= FINAL_CUTOFF) με FINAL_CUTOFF = 32768. Έτσι, όταν το μέγεθος του υποπίνακα πέσει κάτω από αυτό το όριο, περιορίζεται η δημιουργία νέων tasks σε βαθύτερα επίπεδα της αναδρομής. Παράλληλα, το mergeable δίνει στο runtime τη δυνατότητα για επιπλέον βελτιστοποίηση στη διαχείριση των tasks.

// Μεταγλώττιση για το baseline version (B ερώτημα)

```
gcc -O2 -fopenmp mergesort.c -o mergesort_b
```

// Μεταγλώττιση για την έκδοση με final + mergeable (Γ ερώτημα)

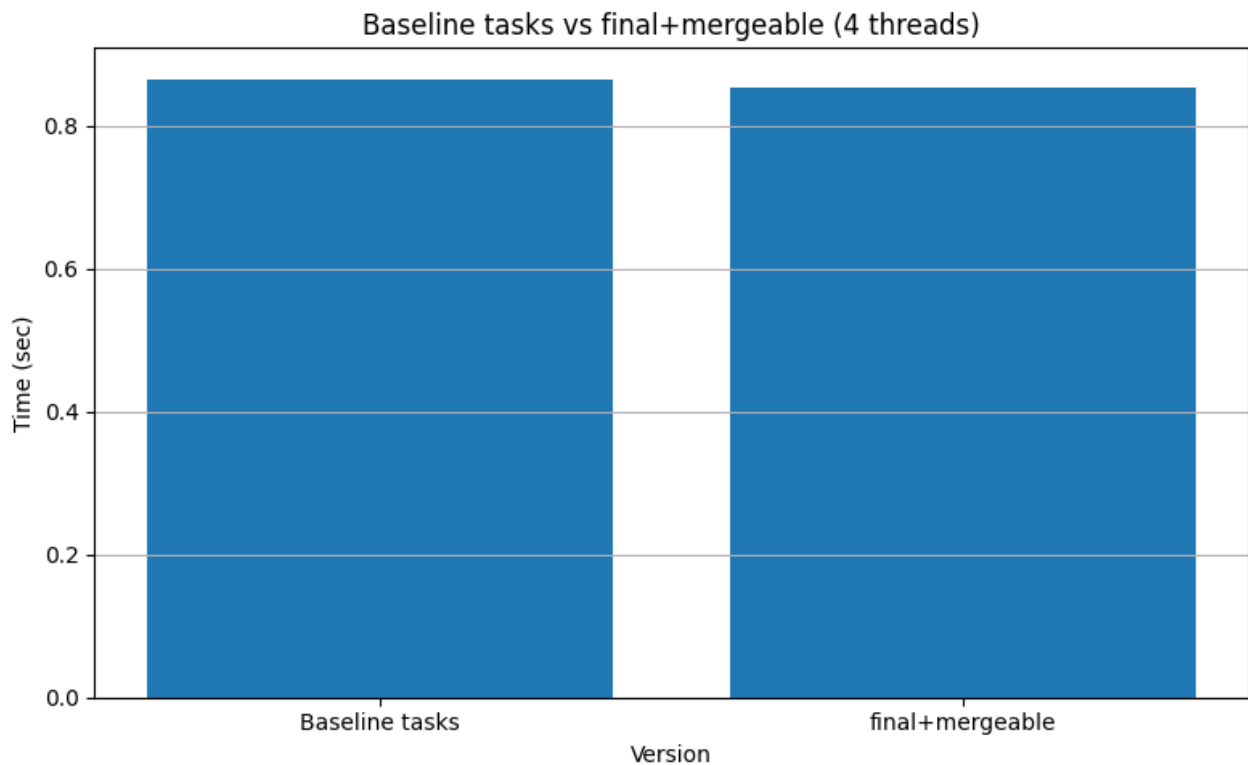
```
gcc -O2 -fopenmp -DUSE_FINAL_MERGEABLE mergesort.c -o mergesort_c
```

Πειραματικά αποτελέσματα – μετρήσεις

Πίνακας 5: Σύγκριση baseline tasks vs final+mergeable – 4 νήματα, N=20M (sec)

Έκδοση	1η εκτ.	2η εκτ.	3η εκτ.	4η εκτ.	Μέσος όρος
Baseline tasks	0.889	0.888	0.840	0.843	0.865
final+mergeable	0.886	0.839	0.841	0.841	0.852

ΓΡΑΦΙΚΗ:



Σχόλια

Από τα αποτελέσματα παρατηρώ ότι η χρήση των final και mergeable οδήγησε σε μια μικρή βελτίωση του χρόνου εκτέλεσης σε σχέση με την baseline έκδοση με tasks. Στα συγκεκριμένα πειράματα, η έκδοση final+mergeable είχε μέσο χρόνο 0.852 sec, ενώ η baseline έκδοση 0.865 sec. Η διαφορά αυτή είναι μικρή, αλλά δείχνει ότι ο περιορισμός της δημιουργίας νέων tasks σε μικρότερα υποπροβλήματα μπορεί να μειώσει λίγο το overhead. Σε αντίθεση με το Β ερώτημα, όπου το κέρδος προέρχεται από την παράλληλη εκτέλεση των δύο υποπροβλημάτων, εδώ το όφελος αφορά μια μικρή μείωση του overhead από τη δημιουργία και διαχείριση των tasks. Έτσι το αποτέλεσμα αυτό είναι λογικό, γιατί στο πρόγραμμα χρησιμοποιείται ήδη cutoff, άρα μέρος του overhead έχει ήδη περιοριστεί και δεν υπάρχει περιθώριο για πολύ μεγάλη επιπλέον βελτίωση.

Συνολική σύγκριση επιτάχυνσης

Πίνακας 6: Speedup (σειριακός / παράλληλος μέσος όρος)

	1 νήμα	2 νήματα	3 νήματα	4 νήματα
Primes (guided)	1.00	2.00	3.00	3.98
Mergesort (tasks)	1.02	1.93	2.14	2.32

Σχόλια:

Από τη συνολική σύγκριση φαίνεται ότι και στις δύο υλοποιήσεις η αύξηση του αριθμού των νημάτων οδηγεί σε μείωση του χρόνου εκτέλεσης και αντίστοιχα σε αύξηση της επιτάχυνσης. Στο πρόβλημα των πρώτων αριθμών, η καλύτερη συμπεριφορά παρατηρήθηκε με την πολιτική `guided`, η οποία πέτυχε speedup περίπου 3.98x στα 4 νήματα. Στο `merge sort` με `tasks`, η επιτάχυνση έφτασε περίπου το 2.32x στα 4 νήματα. Η διαφορά αυτή είναι λογική, επειδή στο `merge sort` το `merge` παραμένει σειριακό και έτσι περιορίζει τη συνολική επιτάχυνση. Τέλος εξέτασα ξεχωριστά την έκδοση `final+mergeable` και έδειξε μια μικρή επιπλέον βελτίωση σε σχέση με την `baseline` έκδοση.